Databases, SQL, NoSQL and FastAPI

A deep, practical guide to database connections

1. Database fundamentals
2. SQL in depth
3. NoSQL in depth
4. SQL vs NoSQL: how to choose
5. How a database connection really works
6. FastAPI with SQL (SQLAlchemy sync and async)
7. Migrations with Alembic
8. FastAPI with MongoDB
9. FastAPI with Redis
10. Security, testing and production checklist
11. Learning roadmap and exercises

1. Database fundamentals

A **database** is an organised collection of data stored so that it can be queried, updated and kept consistent. A **DBMS** (database management system) is the software that does this: PostgreSQL, MySQL, SQLite, MongoDB, Redis and so on. Your application never touches the data files directly. It sends requests to the DBMS over a **connection**.

Why not just use files?

- **Concurrency**: many users reading and writing at once without corrupting data.
- **Durability**: committed data survives crashes and power loss.
- **Querying**: find rows by any condition quickly, using indexes instead of scanning everything.
- **Integrity**: rules (types, uniqueness, foreign keys) are enforced by the database itself.
- **Security**: users, roles and permissions.

The two big families

	Relational (SQL)	Non-relational (NoSQL)
Data model	Tables, rows, columns	Documents, key-value pairs, wide columns, graphs
Schema	Fixed, defined up front	Flexible / schema-less
Query language	SQL (standard)	Differs per database
Relationships	Foreign keys and JOINs	Embedding or manual references
Consistency	ACID transactions	Often BASE / tunable
Scaling	Vertical first; replicas, sharding is harder	Designed for horizontal scaling
Examples	PostgreSQL, MySQL, SQLite, SQL Server, Oracle	MongoDB, Redis, Cassandra, DynamoDB, Neo4j

2. SQL in depth

SQL (Structured Query Language) is the standard language for relational databases. Its statements fall into groups:

Group	Purpose	Statements
DDL	Define structure	CREATE, ALTER, DROP, TRUNCATE
DML	Change data	INSERT, UPDATE, DELETE
DQL	Read data	SELECT
DCL	Permissions	GRANT, REVOKE
TCL	Transactions	BEGIN, COMMIT, ROLLBACK, SAVEPOINT

2.1 Creating tables and constraints

```
CREATE TABLE users (
    id         SERIAL PRIMARY KEY,            -- auto-increment (Postgres)
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(255) NOT NULL UNIQUE,
    age        INT CHECK (age >= 0),
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE orders (
    id       SERIAL PRIMARY KEY,
    user_id  INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    total    NUMERIC(10,2) NOT NULL,
    status   VARCHAR(20) DEFAULT 'pending'
);
```

- **PRIMARY KEY**: uniquely identifies a row.
- **FOREIGN KEY / REFERENCES**: links a row to a row in another table; the DB rejects orphans.
- **UNIQUE, NOT NULL, CHECK, DEFAULT**: integrity rules enforced for every client.

2.2 CRUD

```
INSERT INTO users (name, email, age) VALUES ('Asha', 'asha@example.com', 24);
INSERT INTO users (name, email) VALUES ('Ravi', 'ravi@example.com'), ('Mina', 'mina@example.com');

SELECT id, name FROM users WHERE age > 18 ORDER BY name ASC LIMIT 10 OFFSET 0;

UPDATE users SET age = 25 WHERE email = 'asha@example.com';   -- always use WHERE!
DELETE FROM users WHERE id = 3;                               -- always use WHERE!
```

Warning: UPDATE or DELETE without WHERE changes every row. Run a SELECT with the same WHERE first, and use a transaction when unsure.

2.3 Filtering

```
WHERE age BETWEEN 18 AND 30
WHERE name LIKE 'A%'               -- starts with A  (% = any chars, _ = one char)
WHERE status IN ('pending','paid')
WHERE age IS NULL                  -- never use = NULL
WHERE (age > 18 AND status = 'paid') OR is_admin = true
```

2.4 Aggregation and grouping

```
SELECT COUNT(*) AS total_orders, SUM(total) AS revenue, AVG(total) AS avg_order
FROM orders;

SELECT user_id, COUNT(*) AS n, SUM(total) AS spent
```

```
FROM orders
WHERE status = 'paid'            -- filters rows BEFORE grouping
GROUP BY user_id
HAVING SUM(total) > 1000         -- filters groups AFTER grouping
ORDER BY spent DESC;
```

2.5 Joins

A JOIN combines rows from two tables using a related column.

Join	Result
INNER JOIN	Only rows that match in both tables
LEFT JOIN	All rows from the left table, matches from the right (NULL if none)
RIGHT JOIN	All rows from the right table, matches from the left
FULL OUTER JOIN	All rows from both, matched where possible
CROSS JOIN	Every combination (cartesian product)

```
-- users with their orders
SELECT u.name, o.id AS order_id, o.total
FROM users u
INNER JOIN orders o ON o.user_id = u.id;

-- users even if they have no orders
SELECT u.name, COUNT(o.id) AS order_count
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name;
```

2.6 Subqueries, CTEs and window functions

```
-- subquery
SELECT name FROM users
WHERE id IN (SELECT user_id FROM orders WHERE total > 500);

-- CTE (readable, reusable step)
WITH big_spenders AS (
    SELECT user_id, SUM(total) AS spent FROM orders GROUP BY user_id HAVING SUM(total) > 1000
)
SELECT u.name, b.spent FROM big_spenders b JOIN users u ON u.id = b.user_id;

-- window function: rank each user's orders by total without collapsing rows
SELECT user_id, id, total,
       RANK() OVER (PARTITION BY user_id ORDER BY total DESC) AS rnk
FROM orders;
```

2.7 Indexes

An **index** is a separate sorted structure (usually a B-tree) that lets the database jump to rows instead of scanning the whole table. It speeds reads but costs disk space and slows writes, so index columns you filter, join or sort on.

```
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE UNIQUE INDEX idx_users_email ON users(email);
CREATE INDEX idx_orders_user_status ON orders(user_id, status);   -- composite

EXPLAIN ANALYZE SELECT * FROM orders WHERE user_id = 42;          -- see the query plan
```

- Look for *Seq Scan* on big tables in EXPLAIN; it often means a missing index.
- Composite index order matters: (user_id, status) helps filters on user_id, or user_id and status, but not status alone.

- Wrapping a column in a function (e.g. LOWER(email)) can prevent index use unless you create a functional index.

2.8 Transactions and ACID

A **transaction** groups statements so they all succeed or all fail.

Letter	Meaning	Example
A - Atomicity	All or nothing	Money leaves account A and arrives in B, or neither happens
C - Consistency	Constraints always hold after commit	No negative balance if a CHECK forbids it
I - Isolation	Concurrent transactions do not see each other's half-done work	Two buyers cannot take the last item
D - Durability	Committed data survives a crash	Write-ahead log is flushed to disk

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;      -- or ROLLBACK; to undo everything
```

Isolation levels (weakest to strongest): READ UNCOMMITTED, READ COMMITTED (Postgres default), REPEATABLE READ, SERIALIZABLE. Stronger levels prevent anomalies such as dirty reads, non-repeatable reads and phantom reads, at the cost of more locking or retries.

2.9 Normalization

Normalization removes redundancy so each fact is stored once.

- **1NF**: every column holds a single value (no comma-separated lists), rows are unique.
- **2NF**: every non-key column depends on the whole primary key.
- **3NF**: non-key columns depend only on the key, not on other non-key columns.
- Denormalize deliberately (duplicate data) only when you measure a read-performance need.

Relationships

```
One-to-many   users 1---* orders       (orders.user_id -> users.id)
Many-to-many  students *---* courses   via a join table:
              enrollments(student_id, course_id, PRIMARY KEY (student_id, course_id))
One-to-one    users 1---1 profiles     (profiles.user_id UNIQUE)
```

2.10 Views, stored procedures and other features

- **View**: a saved query that behaves like a table (CREATE VIEW ...).
- **Materialized view**: a view whose result is stored and refreshed on demand.
- **Trigger / stored procedure**: logic that runs inside the database. Powerful but harder to test and version.
- **JSONB** (Postgres): store and index JSON inside a relational table, a bridge to NoSQL ideas.

2.11 Popular SQL databases

Database	Notes
PostgreSQL	Feature-rich, standards-compliant, JSONB, extensions. Best default for new apps.
MySQL / MariaDB	Very widespread, fast for simple reads, huge ecosystem.

Database	Notes
SQLite	Single file, no server. Great for learning, tests, small apps.
SQL Server / Oracle	Enterprise systems, strong tooling, licensed.

3. NoSQL in depth

NoSQL ("not only SQL") covers databases that drop the fixed table model to gain flexibility or scale. There are four main types:

Type	Model	Best for	Examples
Document	JSON-like documents in collections	Content, catalogs, user profiles, general apps	MongoDB, Couchbase, Firestore
Key-value	key -> value	Caching, sessions, counters, queues	Redis, Memcached, DynamoDB
Wide-column	Rows with flexible column families	Huge write-heavy time-series / logs	Cassandra, HBase, ScyllaDB
Graph	Nodes and edges	Social networks, recommendations, fraud rings	Neo4j, ArangoDB

3.1 CAP theorem and BASE

In a distributed system that suffers a network **P**artition, you must choose between **C**onsistency (every read sees the latest write) and **A**vailability (every request gets an answer). Many NoSQL systems choose availability and offer **BASE**: Basically Available, Soft state, Eventually consistent. Several (MongoDB, DynamoDB, Cassandra) let you tune consistency per operation.

3.2 MongoDB (document database)

Terminology: database -> **collection** (like a table) -> **document** (like a row, stored as BSON). Each document has an _id.

```
// mongosh
use shop

db.users.insertOne({
  name: "Asha", email: "asha@example.com",
  address: { city: "Pune", pin: "411001" },        // embedded sub-document
  tags: ["premium", "newsletter"]                  // array
})

db.users.find({ "address.city": "Pune", age: { $gte: 18 } }, { name: 1, email: 1 })
db.users.updateOne({ email: "asha@example.com" }, { $set: { age: 25 }, $push: { tags: "vip" } })
db.users.deleteOne({ email: "asha@example.com" })

db.users.createIndex({ email: 1 }, { unique: true })
```

Common operators

Kind	Operators
Comparison	$eq $ne $gt $gte $lt $lte $in $nin
Logical	$and $or $not $nor
Update	$set $unset $inc $push $pull $addToSet
Aggregation stages	$match $group $sort $project $lookup $unwind $limit

```
// Aggregation pipeline: total spent per user, top 5
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $group: { _id: "$user_id", spent: { $sum: "$total" }, n: { $sum: 1 } } },
  { $sort: { spent: -1 } },
  { $limit: 5 },
  { $lookup: { from: "users", localField: "_id", foreignField: "_id", as: "user" } }
])
```

Embed or reference?

- **Embed** when data is read together and owned by the parent (an order's line items).
- **Reference** (store the other document's id) when data is shared, large or grows without bound (a user's millions of events).
- Design collections around your *queries*, not around normalisation rules.
- Single-document writes are atomic. Multi-document transactions exist (replica sets) but are costlier.

3.3 Redis (key-value / in-memory)

Redis keeps data in memory, so reads and writes take microseconds. It is mostly used for caching, sessions, rate limiting, leaderboards and queues, and typically sits *beside* a primary database rather than replacing it.

```
SET user:42:name "Asha"
GET user:42:name
SET session:abc123 "{...}" EX 3600       # expires in 1 hour
INCR page:views                          # atomic counter
HSET user:42 name Asha age 24            # hash
LPUSH jobs "send_email"                  # list as queue
SADD tags:post:1 python fastapi          # set
ZADD leaderboard 1500 "asha"             # sorted set
ZREVRANGE leaderboard 0 9 WITHSCORES     # top 10
```

Caching pattern (cache-aside)

```
1. App asks Redis for key.
2. HIT  -> return value.
3. MISS -> query the SQL database, store result in Redis with a TTL, return it.
4. On update/delete, invalidate (delete) the cached key.
```

3.4 Cassandra and Neo4j in one paragraph each

Cassandra distributes data across many nodes with no single master and is built for very high write throughput. You model tables per query (denormalised) and choose a partition key carefully. **Neo4j** stores nodes and relationships natively; the Cypher query MATCH (a:User)-[:FOLLOWS]->(b:User) RETURN b traverses relationships without costly JOINs, which suits recommendations and network analysis.

4. SQL vs NoSQL: how to choose

Situation	Pick
Structured data with relationships, reporting, money, inventory	SQL (PostgreSQL)
Need strict transactions across several tables	SQL
Rapidly changing data shape, nested documents	Document DB (MongoDB), or Postgres JSONB
Caching, sessions, rate limiting, real-time counters	Redis
Massive write volume, time-series, multi-region	Cassandra / DynamoDB
Highly connected data and path queries	Graph DB
Search with ranking and fuzzy text	Elasticsearch / OpenSearch (or Postgres full-text)

Practical advice: start with PostgreSQL. It handles most workloads, supports JSON, and is easy to reason about. Add Redis for caching and a specialised store only when you hit a concrete limit. Many production systems are **polyglot**: SQL as the source of truth, Redis as cache, a search engine for text.

5. How a database connection really works

5.1 The life of a connection

```
Your app                                  Database server
   |  1. TCP connect (host:port)  ------------->  |
   |  2. TLS handshake (optional) <-------------> |
   |  3. Authenticate (user/password) ---------->  |
   |  4. Choose database/schema   ------------->  |
   |  5. Send queries, receive rows <---------->  |
   |  6. Close                    ------------->  |
```

Steps 1 to 4 are **expensive** (milliseconds, plus server memory per connection; Postgres forks a process per connection). Opening a new connection per request would waste time and exhaust the server.

5.2 Connection string (URL)

```
dialect+driver://username:password@host:port/database?options

postgresql+psycopg2://app:secret@localhost:5432/shop        (sync)
postgresql+asyncpg://app:secret@localhost:5432/shop         (async)
mysql+pymysql://app:secret@localhost:3306/shop
sqlite:///./app.db                                           (file)
mongodb://user:pass@localhost:27017/shop?authSource=admin
mongodb+srv://user:pass@cluster0.xxxx.mongodb.net/shop       (Atlas)
redis://:password@localhost:6379/0
```

Special characters in passwords (@ : / #) must be URL-encoded, e.g. p@ss becomes p%40ss.

5.3 Drivers

A **driver** speaks the database's wire protocol. An **ORM** (SQLAlchemy, SQLModel, Django ORM) sits on top and maps tables to Python classes.

Database	Sync driver	Async driver
PostgreSQL	psycopg2 / psycopg 3	asyncpg, psycopg 3 async
MySQL	PyMySQL, mysqlclient	aiomysql, asyncmy
SQLite	sqlite3 (built in)	aiosqlite
MongoDB	PyMongo	Motor, PyMongo async, (ODM: Beanie)
Redis	redis-py	redis.asyncio

5.4 Connection pooling

A **pool** keeps a set of open connections and lends them out. Your code "borrows" a connection, uses it and returns it; nothing is re-opened.

```
engine = create_engine(
    DATABASE_URL,
    pool_size=10,         # connections kept open
    max_overflow=20,      # extra temporary connections under load
    pool_timeout=30,      # seconds to wait for a free connection before error
    pool_recycle=1800,    # replace connections older than 30 min (avoids stale ones)
    pool_pre_ping=True,   # test a connection before use; survives DB restarts
)
```

- Total possible connections = pool_size + max_overflow, **per process**. With 4 Uvicorn workers you may open 4 x 30 = 120. Keep this under the database's max_connections (Postgres default 100).
- For many app instances use an external pooler such as **PgBouncer**.

- "QueuePool limit reached" errors usually mean sessions are not being closed (a leak) or the pool is too small.

5.5 Engine vs Connection vs Session (SQLAlchemy)

Object	Role	Lifetime
Engine	Holds the pool and dialect. Created once.	Whole application
Connection	One borrowed DBAPI connection	Short (inside a session/transaction)
Session	Unit of work: tracks objects, flushes changes, manages a transaction	One per request

5.6 Sync vs async

- **Sync** (def routes): simple; FastAPI runs them in a threadpool (default 40 threads), so blocking calls do not freeze the event loop.
- **Async** (async def routes): one thread handles many waiting requests; needs an async driver. Never call blocking DB code inside async def; it blocks every request.
- Async shines under many concurrent, I/O-bound requests. For modest traffic, sync is perfectly fine and simpler.

6. FastAPI with SQL (SQLAlchemy)

6.1 Project layout

```
app/
  core/config.py        # settings from environment
  db/session.py         # engine, SessionLocal, get_db
  db/base.py            # Base class
  models/               # SQLAlchemy tables
  schemas/              # Pydantic request/response models
  crud/                 # database functions
  routers/              # API endpoints
  main.py
```

6.2 Configuration from environment

```
# pip install pydantic-settings
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str = "sqlite:///./app.db"
    class Config:
        env_file = ".env"

settings = Settings()

# .env  (never commit this file)
# DATABASE_URL=postgresql+psycopg2://app:secret@localhost:5432/shop
```

6.3 Sync setup, SQLAlchemy 2.0 style

```
# db/session.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase
from app.core.config import settings

engine = create_engine(settings.database_url, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# models/user.py
from sqlalchemy import String, ForeignKey, Numeric
from sqlalchemy.orm import Mapped, mapped_column, relationship
from app.db.session import Base

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    orders: Mapped[list["Order"]] = relationship(back_populates="user", cascade="all, delete-orphan")

class Order(Base):
    __tablename__ = "orders"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    total: Mapped[float] = mapped_column(Numeric(10, 2))
    user: Mapped["User"] = relationship(back_populates="orders")

# schemas/user.py
from pydantic import BaseModel, EmailStr, ConfigDict
```

```
class UserCreate(BaseModel):
    name: str
    email: EmailStr

class UserOut(BaseModel):
    id: int
    name: str
    email: EmailStr
    model_config = ConfigDict(from_attributes=True)   # read from ORM objects

# routers/users.py
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session
from app.db.session import get_db
from app.models.user import User
from app.schemas.user import UserCreate, UserOut

router = APIRouter(prefix="/users", tags=["users"])

@router.post("", response_model=UserOut, status_code=status.HTTP_201_CREATED)
def create_user(data: UserCreate, db: Session = Depends(get_db)):
    user = User(**data.model_dump())
    db.add(user)
    try:
        db.commit()
    except IntegrityError:            # unique email violated
        db.rollback()
        raise HTTPException(409, "Email already exists")
    db.refresh(user)
    return user

@router.get("", response_model=list[UserOut])
def list_users(skip: int = 0, limit: int = 50, db: Session = Depends(get_db)):
    return db.scalars(select(User).offset(skip).limit(limit)).all()

@router.get("/{user_id}", response_model=UserOut)
def get_user(user_id: int, db: Session = Depends(get_db)):
    user = db.get(User, user_id)
    if not user:
        raise HTTPException(404, "User not found")
    return user

@router.delete("/{user_id}", status_code=204)
def delete_user(user_id: int, db: Session = Depends(get_db)):
    user = db.get(User, user_id)
    if not user:
        raise HTTPException(404, "User not found")
    db.delete(user)
    db.commit()
```

6.4 How Depends(get_db) works

- FastAPI calls get_db() before your route; the value after yield is injected.
- After the response is produced, the code after yield (the finally) runs, closing the session and returning the connection to the pool.
- If the same dependency is used several times in one request, FastAPI reuses one instance (cache per request).

6.5 Querying with SQLAlchemy 2.0

```
from sqlalchemy import select, func, and_, or_
from sqlalchemy.orm import selectinload

# filter + order
stmt = select(User).where(User.email.like("%@example.com")).order_by(User.name)
```

```
users = db.scalars(stmt).all()

# join and aggregate
stmt = (select(User.name, func.count(Order.id), func.sum(Order.total))
        .join(Order, Order.user_id == User.id)
        .group_by(User.id))
rows = db.execute(stmt).all()

# avoid the N+1 problem: load orders in one extra query
stmt = select(User).options(selectinload(User.orders))

# raw SQL, parameterised (safe)
from sqlalchemy import text
db.execute(text("SELECT * FROM users WHERE email = :e"), {"e": email})
```

N+1 problem: loading 100 users then touching user.orders for each runs 101 queries. Use selectinload or joinedload to fetch related rows up front.

6.6 Transactions in practice

```
def transfer(db: Session, src: int, dst: int, amount: float):
    try:
        a = db.get(Account, src, with_for_update=True)   # row lock
        b = db.get(Account, dst, with_for_update=True)
        if a.balance < amount:
            raise ValueError("Insufficient funds")
        a.balance -= amount
        b.balance += amount
        db.commit()                 # both updates become permanent together
    except Exception:
        db.rollback()               # neither is applied
        raise
```

6.7 Async setup

```
# db/session.py (async)
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession

engine = create_async_engine("postgresql+asyncpg://app:secret@localhost/shop",
                             pool_size=10, max_overflow=20, pool_pre_ping=True)
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_db():
    async with AsyncSessionLocal() as session:
        yield session

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

@router.get("/{user_id}", response_model=UserOut)
async def get_user(user_id: int, db: AsyncSession = Depends(get_db)):
    user = await db.get(User, user_id)
    if not user:
        raise HTTPException(404, "User not found")
    return user

@router.get("", response_model=list[UserOut])
async def list_users(db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User).limit(50))
    return result.scalars().all()
```

- With async, lazy loading of relationships raises errors. Load them eagerly with selectinload.
- Use await engine.dispose() on shutdown (see lifespan below).

6.8 Lifespan: startup and shutdown

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # startup: e.g. verify DB is reachable
    async with engine.begin() as conn:
        await conn.run_sync(lambda c: None)
    yield
    # shutdown: release pool
    await engine.dispose()

app = FastAPI(lifespan=lifespan)
```

6.9 SQLModel: less boilerplate

SQLModel (from FastAPI's author) merges SQLAlchemy tables and Pydantic models into one class.

```
from sqlmodel import SQLModel, Field, Session, create_engine, select

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    age: int | None = None

engine = create_engine("sqlite:///heroes.db")
SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

@app.post("/heroes")
def create(hero: Hero, session: Session = Depends(get_session)):
    session.add(hero); session.commit(); session.refresh(hero)
    return hero
```

7. Migrations with Alembic

A **migration** is a versioned script that changes the database structure. It lets the whole team (and production) evolve the schema safely, and you can roll back. create_all() only creates missing tables; it cannot alter existing ones.

```
pip install alembic
alembic init migrations                 # creates alembic.ini and migrations/

# migrations/env.py
from app.db.session import Base
from app.models import user             # import models so they register
target_metadata = Base.metadata
# set sqlalchemy.url (from settings) in env.py or alembic.ini

alembic revision --autogenerate -m "create users and orders"   # generate script
alembic upgrade head                    # apply all migrations
alembic downgrade -1                    # undo last one
alembic history                         # list revisions
alembic current                         # which revision is applied
```

- Always **review** autogenerated scripts; renames can appear as drop + create (data loss).
- Make risky changes in steps: add nullable column, backfill, then add NOT NULL.
- Run alembic upgrade head as a deploy step, not at app import time.

This repository already contains alembic.ini and a migrations/ folder, so you can practise on real project structure.

8. FastAPI with MongoDB

8.1 Motor (async driver)

```
# pip install motor
from motor.motor_asyncio import AsyncIOMotorClient
from contextlib import asynccontextmanager
from fastapi import FastAPI, Depends, HTTPException, Request
from bson import ObjectId
from pydantic import BaseModel, EmailStr

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.client = AsyncIOMotorClient("mongodb://localhost:27017", maxPoolSize=50)
    app.state.db = app.state.client["shop"]
    await app.state.db.users.create_index("email", unique=True)
    yield
    app.state.client.close()

app = FastAPI(lifespan=lifespan)

def get_db(request: Request):
    return request.app.state.db

class UserIn(BaseModel):
    name: str
    email: EmailStr

def to_out(doc):
    doc["id"] = str(doc.pop("_id"))      # ObjectId is not JSON serialisable
    return doc

@app.post("/users", status_code=201)
async def create(user: UserIn, db=Depends(get_db)):
    res = await db.users.insert_one(user.model_dump())
    return {"id": str(res.inserted_id)}

@app.get("/users/{uid}")
async def read(uid: str, db=Depends(get_db)):
    if not ObjectId.is_valid(uid):
        raise HTTPException(422, "Invalid id")
    doc = await db.users.find_one({"_id": ObjectId(uid)})
    if not doc:
        raise HTTPException(404, "Not found")
    return to_out(doc)

@app.get("/users")
async def list_(skip: int = 0, limit: int = 20, db=Depends(get_db)):
    cursor = db.users.find().skip(skip).limit(limit)
    return [to_out(d) async for d in cursor]
```

- Create **one** client for the whole app (it holds its own connection pool); never one per request.
- ObjectId must be converted to str for JSON.
- Beanie is an async ODM on top of Motor that gives Pydantic-style document classes with await User.find(...).

9. FastAPI with Redis

9.1 Connection and caching

```
# pip install redis
import json
import redis.asyncio as redis
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request, Depends

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.redis = redis.from_url("redis://localhost:6379/0",
                                     decode_responses=True, max_connections=20)
    yield
    await app.state.redis.aclose()

app = FastAPI(lifespan=lifespan)

def get_redis(request: Request):
    return request.app.state.redis

@app.get("/users/{user_id}")
async def get_user(user_id: int, r=Depends(get_redis), db=Depends(get_db)):
    key = f"user:{user_id}"
    cached = await r.get(key)
    if cached:
        return json.loads(cached)                       # cache HIT
    user = await db.get(User, user_id)                  # cache MISS -> SQL
    if not user:
        raise HTTPException(404)
    data = {"id": user.id, "name": user.name}
    await r.set(key, json.dumps(data), ex=300)          # TTL 5 minutes
    return data

@app.put("/users/{user_id}")
async def update_user(user_id: int, r=Depends(get_redis)):
    ...                                                 # update SQL, then:
    await r.delete(f"user:{user_id}")                   # invalidate cache
```

9.2 Rate limiting with INCR

```
async def rate_limit(request: Request, r=Depends(get_redis)):
    key = f"rl:{request.client.host}"
    count = await r.incr(key)
    if count == 1:
        await r.expire(key, 60)           # window of 60 seconds
    if count > 100:
        raise HTTPException(429, "Too many requests")

@app.get("/search", dependencies=[Depends(rate_limit)])
async def search(): ...
```

10. Security, testing and production checklist

10.1 SQL injection

SQL injection happens when user input is glued into a SQL string, letting an attacker change the query's meaning.

```
# DANGEROUS: string formatting
email = request_param            # "x' OR '1'='1"
db.execute(text(f"SELECT * FROM users WHERE email = '{email}'"))   # returns ALL users

# SAFE: bound parameters (driver sends data separately from the SQL)
db.execute(text("SELECT * FROM users WHERE email = :e"), {"e": email})

# SAFE: ORM / expression language builds parameters for you
db.scalars(select(User).where(User.email == email))
```

MongoDB has the same class of problem (NoSQL injection): never pass raw user-supplied JSON as a query filter. Validate inputs with Pydantic models and reject operator keys like $ne or $gt in user data.

10.2 Other security rules

- Use a dedicated DB user with **least privilege** (no superuser for the app).
- Keep credentials in environment variables or a secrets manager, never in git.
- Use TLS to the database in production (?sslmode=require).
- Hash passwords with bcrypt or argon2 before storing; never store plain text.
- Do not expose database ports to the internet; use private networks.
- Never return ORM models containing password hashes; use response_model schemas.

10.3 Testing with a database

```
# conftest.py - use an in-memory/temp database and override the dependency
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool
from app.main import app
from app.db.session import Base, get_db

engine = create_engine("sqlite://", connect_args={"check_same_thread": False}, poolclass=StaticPool)
TestingSession = sessionmaker(bind=engine)

@pytest.fixture()
def client():
    Base.metadata.create_all(engine)
    def override():
        db = TestingSession()
        try: yield db
        finally: db.close()
    app.dependency_overrides[get_db] = override
    yield TestClient(app)
    app.dependency_overrides.clear()
    Base.metadata.drop_all(engine)

def test_create_user(client):
    r = client.post("/users", json={"name": "Asha", "email": "a@x.com"})
    assert r.status_code == 201
```

For fidelity with production, test against the real engine (PostgreSQL in Docker or Testcontainers); SQLite differs in types, locking and some SQL features.

10.4 Docker Compose for local databases

```
services:
  postgres:
    image: postgres:16
    environment: { POSTGRES_USER: app, POSTGRES_PASSWORD: secret, POSTGRES_DB: shop }
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
  mongo:
    image: mongo:7
    ports: ["27017:27017"]
  redis:
    image: redis:7
    ports: ["6379:6379"]
volumes:
  pgdata:
```

10.5 Production checklist

Area	Check
Pooling	pool_size x workers below DB max_connections; pool_pre_ping on; consider PgBouncer
Sessions	One per request, always closed (dependency with try/finally)
Migrations	Alembic only; reviewed; run on deploy
Indexes	On foreign keys and frequently filtered columns; verify with EXPLAIN
Queries	No N+1; paginate all list endpoints; select only needed columns
Timeouts	Statement and pool timeouts so slow queries do not pile up
Backups	Automated, tested restores; point-in-time recovery for critical data
Monitoring	Slow query log, connection count, replication lag, disk space
Secrets	Env vars / vault; rotate credentials
Errors	Handle IntegrityError, OperationalError; return proper 4xx/5xx

10.6 Common mistakes

- Creating an engine or client per request (connection storm).
- Forgetting db.commit(), so changes silently vanish.
- Using blocking DB calls inside async def.
- Holding a transaction open while calling slow external APIs.
- Returning unbounded lists (no LIMIT).
- Storing money as float; use NUMERIC/Decimal.
- Relying on create_all in production instead of migrations.

11. Learning roadmap and exercises

Suggested order

- **Week 1: SQL basics.** Install SQLite or PostgreSQL; practise CREATE, INSERT, SELECT, WHERE, ORDER BY.
- **Week 2: Relational design.** Joins, GROUP BY, constraints, normalization, indexes, EXPLAIN.
- **Week 3: Transactions and Python access.** ACID, isolation, psycopg2 / sqlite3, then SQLAlchemy.
- **Week 4: FastAPI + SQL.** CRUD API, Depends(get_db), Pydantic schemas, Alembic migrations.
- **Week 5: NoSQL.** MongoDB shell and aggregation; Redis data types and caching.
- **Week 6: Production skills.** Async, pooling, testing, Docker Compose, security.

Hands-on exercises

- **1.** Model a blog: users, posts, comments, tags (many-to-many). Write queries for 'top 5 authors by comment count'.
- **2.** Build the FastAPI CRUD API for the blog with SQLAlchemy and Alembic migrations.
- **3.** Add pagination, filtering by tag, and an index; compare EXPLAIN before and after.
- **4.** Add Redis caching of the post list with a 60-second TTL and invalidate on write.
- **5.** Re-implement 'posts with embedded comments' in MongoDB and compare the design.
- **6.** Write pytest tests with dependency overrides; then run them against PostgreSQL in Docker.
- **7.** Simulate a pool exhaustion (pool_size=1, slow endpoint) and observe the timeout error.

Quick glossary

Term	Meaning
Schema	The structure of a database (tables, columns, types)
Index	Sorted lookup structure that speeds up reads
Transaction	A group of operations that commit or roll back together
ORM	Maps tables/rows to classes/objects
ODM	Same idea for document databases
Pool	Reusable set of open connections
Migration	Versioned schema change script
Sharding	Splitting data across servers by a key
Replication	Copying data to other servers for availability and read scaling
TTL	Time to live: automatic expiry of a key or document
N+1	One query to list plus one query per item; fix with eager loading

Tip: keep experimenting. Reading explains the ideas, but you will only truly learn them by building small projects and breaking them.